

操作系统课程设计，离散化 Unix V6++ 系统

2351520 毛星博 计算机拔尖班

实验目标

UNIX V6++ 离散页式存储管理改造：将 UNIX V6++ 系统改为离散的存储管理方式，页式。不使用盘交换区。

实验步骤

1：分配器从连续分配扩展到位图分配

内存管理系统维护 1 张位示图。位图中每个 bit 对应一个物理页框，4096 字节；0 表示空闲，1 表示已分配。

新增 BitMap 结构（记录页池起始地址、位图行数、位图数组）。

```
class BitMap
{
public:
    unsigned long m_AddressIdx; // 分配空间的起始地址，不变
    int rows;
    unsigned long long int map[256]; // TODO: 实现大小根据需要开辟
    void set(unsigned long startAddr, unsigned long page_num)
    {
        m_AddressIdx = startAddr;
        rows = page_num / 64;
        for (int i = 0; i < rows; i++)
        {
            map[i] = 0;
        }
    };
};
```

新增 BitMapAllocator 类

```
class BitMapAllocator
{
    /* Functions */
public:
    unsigned long Alloc(BitMap &bitmap, unsigned long size);
    unsigned long Free(BitMap &bitmap, unsigned long size, unsigned long addrIdx);
    int is_free(BitMap &bitmap, int index);
    void set_bit(BitMap &bitmap, int index, int value);

public:
    static BitMapAllocator &GetInstance();
};
```

```
private:
    static BitMapAllocator m_Instance_bitmap;
};
```

Kernel 中页管理器实例改为使用 BitMapAllocator

```
UserPageManager g_UserPageManager(&(BitMapAllocator::GetInstance()));
KernelPageManager g_KernelPageManager(&(BitMapAllocator::GetInstance()));
```

2: PageManager 引入页级引用计数

PageManager 构造函数参数由 Allocator* 改为 BitMapAllocator*。

```
PageManager(BitMapAllocator *allocator);
```

新增 BitMap bitmap 与 int Page[]（页引用计数）。

```
// NOTE:1
BitMap bitmap;

// NOTE:3
int Page[MemoryDescriptor::USER_SPACE_SIZE / PAGE_SIZE];
```

AllocMemory(): 分配后将页引用计数置为 1。

FreeMemory(): 先递减引用计数，计数归零才真正回收页框。

```
unsigned long PageManager::AllocMemory(unsigned long size)
{
    unsigned long newaddr = this->m_pAllocator->Alloc(this->bitmap, size);
    Page[newaddr >> 12] = 1;
    return newaddr;
}

unsigned long PageManager::FreeMemory(unsigned long size, unsigned long startAddress)
{
    Page[startAddress >> 12] --;
    if(Page[startAddress >> 12]==0)
        this->m_pAllocator->Free(this->bitmap, size, startAddress);
    return 1;
}
```

3: 页表刷新机制改为“按进程页目录地址刷新”

FlushPageDirectory() 宏由固定写 cr3 = 0x200000 改为接收参数（动态页目录物理地址）。

SwthUStruct、MapToPageTable、PEParser::Relocate 等处配套改用“当前进程页目录”刷新。

4: “0 公用页表 + 进程私有页目录”初始化

Machine 新增 InitKernelPageTable()、InitZeroUserPageTable()。

启用分页时直接使用固定页目录物理基址；运行期依赖进程切换时刷新到对应 CR3。

启动流程里不再简单调用旧的 InitUserPageTable() 主路径，而是转向“给 1 进程写私有页表并刷新页目录”。将“全局用户页表”模型调整为“进程私有地址空间”模型。

```
void Machine::InitKernelPageTable()
{
    // 核心页表
    PageTable *pkernelPageTable = (PageTable *) (KERNEL_PAGE_TABLE_BASE_ADDRESS +
    KERNEL_SPACE_START_ADDRESS);
    for (unsigned int i = 0; i < PageTable::ENTRY_CNT_PER_PAGETABLE; i++)
    {
        pkernelPageTable->m_Entrys[i].m_UserSupervisor = 0;
        pkernelPageTable->m_Entrys[i].m_Present = 1;
        pkernelPageTable->m_Entrys[i].m_ReadWriter = 1;
        pkernelPageTable->m_Entrys[i].m_PageBaseAddress = i;
    }
    this->m_KernelPageTable = pkernelPageTable;
}

void Machine::InitZeroUserPageTable()
{
    // 0#用户页表
    PageTable *pUserPageTable = (PageTable *) (USER_PAGE_TABLE_BASE_ADDRESS +
    KERNEL_SPACE_START_ADDRESS);
    for (unsigned int i = 0; i < PageTable::ENTRY_CNT_PER_PAGETABLE; i++)
    {
        pUserPageTable->m_Entrys[i].m_UserSupervisor = 1;
        pUserPageTable->m_Entrys[i].m_Present = 1;
        pUserPageTable->m_Entrys[i].m_ReadWriter = 1;
        pUserPageTable->m_Entrys[i].m_PageBaseAddress = 0x00000 + i;
    }
    this->m_UserPageTable = pUserPageTable;
}
```

5: MemoryDescriptor 从“两张页表整体映射”转向“单页表 + 进程切换驱动”

Initialize/Release 从申请/释放 sizeof(PageTable) * USER_SPACE_PAGE_TABLE_CNT 改为仅一张页表。

MapToPageTable() 改为按当前进程页目录刷新。

ClearUserPageTable() 从双页表循环改为单页表清理。

6: fork 路径改为 COW, 不再复制整段镜像

新增 ProcessManager::ModifyPageTable(UserPageManager&, PageTable*):

```
void ModifyPageTable(UserPageManager &userPageManager, PageTable* pgTable);
```

将父进程可写页改为只读；

对相应物理页引用计数 Page[]++。

NewProc() 中：

子进程仅为 PPDA 分配一页；

复制父页表（单页表）；

不再按 current->p_size 整段复制进程镜像。

fork 成本从“按进程镜像大小复制”变为“先共享页 + 写时再复制”。

7: 缺页异常增加 COW 分支

在 Exception::PageFault() 中新增逻辑：

命中条件：访问数据/栈区且页只读，且引用计数有效。

若引用计数为 1：直接将该页 PTE 改回可写。

若引用计数 > 1：

为当前进程分配新物理页；

修改当前进程 PTE 指向新页并设可写；

旧页引用计数减 1；新页引用计数置 1；

用 Utility::CopySeg 拷贝页内容。

把 fork 后的数据拷贝推迟到真正写入时发生。

```
// ---- NOTE:COW ---- //
unsigned char RW = md.m_UserPageTableArray->m_Entrys[(cr2 >> 12) - 1024].m_ReadWriter;

// 思路：需要处理 COW 的情况是
// 缺页异常发生在数据段或者堆栈段 &
// R/W 位内容为 Read Only &
// 引用计数器 (Page) >= 1
Process *pcurrent = NULL;
UserPageManager &userPageMgr = Kernel::Instance().GetUserPageManager();
ProcessManager &procMgr = Kernel::Instance().GetProcessManager();
bool isRW = (cr2 >= md.m_DataStartAddress && cr2 < md.m_DataStartAddress + md.m_DataSize) ||
            (cr2 >= MemoryDescriptor::USER_SPACE_SIZE - md.m_StackSize && cr2 <
MemoryDescriptor::USER_SPACE_SIZE);
unsigned int physicalBase = md.m_UserPageTableArray->m_Entrys[(cr2 >> 12) -
1024].m_PageBaseAddress;
// ! 子进程和父进程没有区别，任意先上台的进程，申请新的空间，复制原本的内容使用
// ! 如果是子进程，则是申请新空间，指向新空间
// ! 如果是父进程，则可以理解为，将原本的空间让给子进程，自己利用新申请的空间
if (RW == 0 && userPageMgr.Page[physicalBase] >= 1 && isRW)
{
    if (userPageMgr.Page[physicalBase] == 1)
    {
        md.m_UserPageTableArray->m_Entrys[(cr2 >> 12) - 1024].m_ReadWriter = 1;
        return;
    }
}
```

```

    }
    else
    {
        --userPageMgr.Page[physicalBase];
        unsigned long newPage = userPageMgr.AllocMemory(M_PAGE_SIZE);
        md.m_UserPageTableArray->m_Entrys[(cr2 >> 12) - 1024].m_PageBaseAddress = newPage >>
12;

        md.m_UserPageTableArray->m_Entrys[(cr2 >> 12) - 1024].m_UserSupervisor = 0x1;
        md.m_UserPageTableArray->m_Entrys[(cr2 >> 12) - 1024].m_Present = 0x1;
        md.m_UserPageTableArray->m_Entrys[(cr2 >> 12) - 1024].m_ReadWriter = 1;
        userPageMgr.Page[newPage >> 12] = 1;
        for (int i = 0; i < procMgr.NPROC; ++i)
        {
            if (procMgr.process[i].p_pid == current->p_ppid)
            {
                pcurrent = &procMgr.process[i];
                break;
            }
        }
        Utility::CopySeg(physicalBase << 12, newPage);
        return;
    }
}

// ---- End of NOTE:COW ---- //

```

8：进程扩缩容从连续镜像迁移到逐页策略

SStack() 改为：

- 每次只分配一页；
- 直接写对应 PTE；
- 刷新页目录。

SBreak() 改为：

- 按页增减数据段 PTE；
- 增长时逐页分配，收缩时逐页释放。

9：Exec/Text 装载路径改为按页操作

Text.x_caddr 从单个地址改为页框数组（按页记录正文段物理页）。

Exec() 中：正文段、数据段、栈段均按页分配并逐项填 PTE。

PEParser::Relocate() 读取页表来源从全局页表改到当前进程页表，并按新下标规则调整正文段读写位。

Text::XccDec() 改为按页释放并清理 PTE。

实验结论

已实现位图分配 + 页表逐项维护 + 逐页扩缩容 + COW，但并未移除所有 swap 调用。
后续工作需要在 Exit()/Exec() 中与 text/u 区 swap 副本相关流程改为纯内存/文件回收策略。

实验截图

虚拟机可以正常打开，各种命令也可以正常执行，说明离散化成功



